B.Tech Computer Science and Engineering (AI & ML) Academic Mini Project

RoadVision

Intelligent Road Monitoring System

Technical Design Document (TDD)

Design of a locally hosted road-damage detection system using deep learning and computer vision.

Document version	1.1 (revised after technical review; aligned with PRD v1.1; supersedes 1.0)
Project	B.Tech CSE (AI & ML) Mini Project
Team size	2 developers (Ram, Shakib)
Development period	7 days
Primary / fallback model	YOLO26m / YOLO26n
Dataset	Complete RDD2022 acquisition; measured training subset
Target platform	Windows 11, NVIDIA RTX 5060 Laptop GPU, 16 GB RAM
Institution	_____
Project guide	_____
Date	November 2026

Legend: [REV] revised from v1.0; [NEW] added in v1.1. Code listings are design proposals: they were syntax-checked but not executed against RDD2022, YOLO26 or the project GPU, and must be tested during implementation. Values marked "recommended" require team approval. Appendix A lists all changes.

Table of Contents

1. Introduction [REV] 4

2. System Architecture [REV] 4

2.1 Serving and process model [NEW] . . . 4

3. Model Architecture and Strategy [REV] 5

3.1 Primary model: YOLO26m . . . 5

3.2 Fallback model: YOLO26n . . . 5

3.3 Model loading and fallback logic [REV] . . . 5

3.4 End-to-end (NMS-free) mode [NEW] . . . 6

4. Dataset Engineering [REV] 7

4.1 Dataset acquisition and integrity [REV] . . . 7

4.2 Dataset inspection . . . 7

4.3 Annotation conversion [REV] . . . 7

4.4 Dataset splitting and leakage prevention [REV] . . . 8

4.5 YOLO dataset structure [REV] . . . 9

4.6 Training subset strategy [NEW] . . . 9

5. Model Training Pipeline [REV] 9

5.1 Training considerations . . . 10

5.2 Staged smoke test and decision rules [NEW] . . . 10

6. Inference Engine [REV] 11

7. Image Processing Module [REV] 11

8. Video Processing Module [REV] 11

9. Live Webcam Module [REV] 13

10. Backend API Design [REV] 14

10.1 Concurrency, jobs, progress and cancellation [REV] . . . 14

10.2 Upload handling, validation and managed files [NEW] . . . 14

10.3 Error shape and schemas [NEW] . . . 15

11. Database Design [REV] 16

12. Frontend Design [REV] 17

13. Proposed Project Folder Structure [REV] 18

14. Hardware and Software Requirements [REV] 18

15. Error Handling and Security [REV] 19

16. Performance, Evaluation and Testing [REV] 20

16.1 Testing strategy [NEW] . . . 20

17. Seven-Day Technical Milestones [REV] 20

18. Licensing and References [REV] 21

Appendix A: Change Log (v1.0 to v1.1) [NEW] 22

Appendix B: Decisions for the Team to Approve [NEW] **22**

Appendix C: Items Not Verified [NEW] **22**

1. Introduction [REV]

RoadVision is a locally hosted road-damage detection system using deep learning and computer vision. It accepts images, recorded videos and live webcam input. Its detection service uses YOLO26m as the primary model and YOLO26n as a lightweight fallback. The system provides annotated outputs, class-wise summaries and local inspection history.

This document describes the intended technical design and implements the requirements of PRD v1.1 (including FR-10 and the operating limits in PRD Section 4.11). It is a design specification: it does not claim that any component has been implemented, and it contains no measured results. Folder names, endpoint names and schema fields are proposals that may be refined during implementation, provided the behaviour described here is preserved.

2. System Architecture [REV]

The application follows a modular client-server architecture with every component running on the local computer, in one backend process.

#	Component	Technology
1	Frontend	React, TypeScript, Vite and Tailwind CSS
2	Backend	Python, FastAPI and Uvicorn (single worker, bound to 127.0.0.1)
3	Detection engine	Ultralytics YOLO26m and YOLO26n on PyTorch
4	Image/video processing	OpenCV; FFmpeg (executable from the imageio-ffmpeg package) for H.264 video output
5	Database	SQLite (WAL mode)
6	File storage	Local directories for temporary uploads and generated outputs
7	Hardware acceleration	CUDA-enabled PyTorch build that supports the RTX 5060 (compute capability 12.0); see Section 14

Table 1: System components

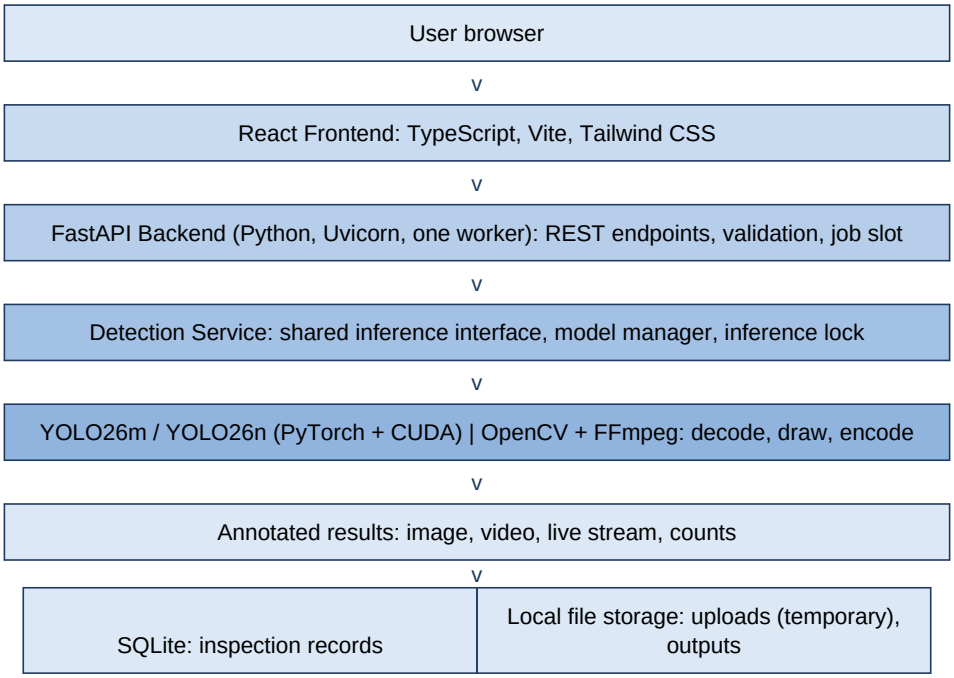


Figure 1: System architecture. The backend persists records and files; results return to the frontend.

2.1 Serving and process model [NEW]

- **Development:** Vite dev server with a proxy that forwards /api to http://127.0.0.1:8000, so CORS is not needed.
- **Demonstration:** the frontend is built (npm run build) and served by FastAPI as static files: one process, one port, one start command.

- **One Uvicorn worker, no auto-reload during demonstrations.** The loaded model, the job slot and the camera are in-process state; multiple workers would duplicate them and exhaust GPU memory.
- **Threading:** handlers that decode media or run inference are plain def functions (executed in FastAPI's thread pool) so the event loop stays free and /api/health, /api/model/status and /api/history respond during processing. All forward passes go through one lock (Section 10.1).

3. Model Architecture and Strategy [REV]

3.1 Primary model: YOLO26m

- Use YOLO26m as the main detection model.
- Start from the official pretrained weights (yolo26m.pt).
- Fine-tune using the prepared RDD2022 dataset.
- Train for the four RoadVision classes (class names are the codes D00, D10, D20, D40).
- Select the checkpoint and confidence threshold on the validation set only.
- Evaluate once on the held-out labelled test set.
- Save the best checkpoint as best.pt and copy it to weights/yolo26m/.

3.2 Fallback model: YOLO26n

- Use YOLO26n as the lightweight alternative; fine-tune it from yolo26n.pt for exactly the same four classes.
- Use identical class names and ordering; save its own checkpoint separately.
- Train it **first** as a pilot (Section 5): it validates the whole pipeline and guarantees that a trained fallback exists.
- Verify that it independently performs road-damage detection on image, video and webcam input and is evaluated on the same test split.

A generic COCO-pretrained YOLO26n checkpoint is not a valid fallback for RoadVision, because it was not trained on road-damage classes. YOLO26n is reported as operational only after it has been trained, smoke-tested in all three input modes and evaluated.

3.3 Model loading and fallback logic [REV]

At start-up the model manager loads YOLO26m, checks that the checkpoint exists and that the model's class names equal the shared class list in order, and runs a warm-up inference. If any step fails, the same sequence runs for YOLO26n. If that also fails, detection endpoints return 503 and the interface shows a clear error; history and dashboard stay available. During operation the triggers below apply (identical to PRD v1.1 Section 4.5).

#	Trigger	Behaviour
F1	YOLO26m checkpoint missing, corrupt or fails to load	Start with YOLO26n; log; show notice
F2	YOLO26m warm-up inference or class check fails at start-up (for example a CUDA error)	Start with YOLO26n; log; show notice
F3	Image inference error (including CUDA out-of-memory)	Retry once on YOLO26m; then switch to YOLO26n and process the request; show notice
F4	Error during a video job	Job restarts once from the beginning on YOLO26n; one model per output video; if YOLO26n also fails the job is marked failed
F5	Live inference on YOLO26m stays below the minimum live FPS (recommended 5) for 10 s	Switch the session to YOLO26n with a visible notice; record both models
F6	Manual selection (optional)	Use the selected model

The system must not switch models silently, and it must not treat a low confidence score or an empty result as evidence of failure: failure is determined by loading, class-check, warm-up and runtime errors, and by measured throughput. After a fallback the application stays on YOLO26n until restart or an explicit re-check. Every fallback is logged with its reason and stored in the inspection's models_used field.

```
# backend/app/services/model_manager.py
import logging, threading
import numpy as np
import torch
```

```
from pathlib import Path
from ultralytics import YOLO
from app.config import CLASS_CODES, MODEL_PATHS, DEVICE, IMGSZ, NMS_ARGS
# NMS_ARGS = {} (end-to-end default) or {"end2end": False, "iou": 0.7}

log = logging.getLogger("roadvision.model")
inference_lock = threading.Lock()          # one forward pass at a time on the GPU

class ModelUnavailable(Exception):
    pass

class ModelManager:
    ORDER = ("YOLO26m", "YOLO26n")
    def __init__(self):
        self.model = None; self.active = None; self.errors = {}; self.switched = False
        self.models_used = []

    def _load(self, name):
        path = Path(MODEL_PATHS[name])
        if not path.is_file():
            raise FileNotFoundError(path)
        m = YOLO(str(path))
        names = [m.names[i] for i in sorted(m.names)]
        if names != CLASS_CODES:
            raise ValueError(f"class mismatch: {names} != {CLASS_CODES}")
        m.predict(np.zeros((IMGSZ, IMGSZ, 3), np.uint8), device=DEVICE, imgsz=IMGSZ, verbose=False, **NMS_ARGS)
        return m

    def startup(self, start_from="YOLO26m"):
        for name in self.ORDER[self.ORDER.index(start_from):]:
            try:
                self.model, self.active = self._load(name), name
                self.models_used = [name]
                if name != "YOLO26m":
                    self.switched = True
                    log.error("FALLBACK ACTIVE: %s (errors: %s)", name, self.errors)
                return
            except Exception as e:                      # load, class check or warm-up failed
                self.errors[name] = repr(e)
                log.exception("model %s not usable", name)
        self.model = self.active = None

    def switch_to_fallback(self, reason):
        if self.active != "YOLO26m":
            return False
        log.error("Switching to YOLO26n: %s", reason)
        self.errors["YOLO26m"] = reason
        self.startup(start_from="YOLO26n")
        return self.model is not None

    def predict(self, frame, conf):
        if self.model is None:
            raise ModelUnavailable("no model available")
        for attempt in range(3):
            try:
                with inference_lock:
                    return self.model.predict(frame, conf=conf, imgsz=IMGSZ, device=DEVICE,
                                              verbose=False, **NMS_ARGS)[0]
            except RuntimeError as e:                   # includes CUDA out-of-memory
                log.exception("inference failed (attempt %d)", attempt + 1)
                if attempt == 0:
                    torch.cuda.empty_cache(); continue   # one retry on the same model
                if self.active == "YOLO26m" and self.switch_to_fallback(repr(e)):
                    continue                            # PRD F3: retry on the fallback
                raise ModelUnavailable(str(e)) from e
        raise ModelUnavailable("inference failed")

def to_result(res, model_name):
    b = res.boxes
    dets = [{"class_id": int(c), "class_name": CLASS_CODES[int(c)], "confidence": round(float(s), 4),
             "box": [round(float(v), 1) for v in xy]}
            for c, s, xy in zip(b.cls.tolist(), b.conf.tolist(), b.xyxy.tolist())]
    counts = {k: 0 for k in CLASS_CODES}
    for d in dets:
        counts[d["class_name"]] += 1
    return {"model": model_name, "total_detections": len(dets), "class_counts": counts, "detections": dets}
```

3.4 End-to-end (NMS-free) mode [NEW]

YOLO26 produces end-to-end, NMS-free predictions by default; an NMS IoU threshold applies only if the traditional path is selected (end2end=False). The team chooses one mode once (recommended: the default end-to-end mode) and uses it for both models, in the application and in evaluation. The mode and, where relevant, the IoU threshold are stored in every inspection record and every evaluation result.

4. Dataset Engineering [REV]

4.1 Dataset acquisition and integrity [REV]

The complete RDD2022 dataset is the acquisition target. The original dataset is downloaded and preserved, and original annotations are never modified; all conversion writes to separate directories.

Item	Reference
Official source	https://doi.org/10.6084/m9.figshare.21431547.v1
Research paper	https://doi.org/10.1002/gdj3.260

Table 2: Dataset sources

- Compare each downloaded file's size and MD5 with the figshare file listing (verify the listing on the page); test archives with Python's zipfile testzip().
- Extract to a short path to avoid the Windows path-length limit; confirm free disk space for the archive, the extracted copy and the converted copy.
- Record file counts before and after extraction. Keep the original archive untouched.

4.2 Dataset inspection

An inspection script (inspect_dataset.py) will: inspect dataset folders; count images and annotation files; identify missing annotations; inspect class distribution (overall and per country or source folder); identify exact duplicates and unreadable files; record image dimensions; and generate a dataset summary. The design does not assume that all downloaded images are labelled or suitable for training; the real class distribution decides the subset strategy (Section 4.6).

4.3 Annotation conversion [REV]

RDD2022 training annotations use Pascal VOC XML. The conversion script (convert_annotations.py) reads each XML file, maps the four selected classes, converts boxes to YOLO's normalised format, writes label files to a separate directory and produces a conversion report. The class mapping is fixed:

Class ID	Original code	Display label
0	D00	Longitudinal crack
1	D10	Transverse crack
2	D20	Alligator crack
3	D40	Pothole

Table 3: Fixed class mapping (single source of truth: classes.json; model class names are the codes)

Each YOLO label line is: class ID, centre x, centre y, width, height, all normalised by the image size. For corners (xmin, ymin), (xmax, ymax) in an image of width W and height H: x_center = ((xmin + xmax) / 2) / W; y_center = ((ymin + ymax) / 2) / H; width = (xmax - xmin) / W; height = (ymax - ymin) / H.

Validation rules: the converter reads each image to obtain its true size and only logs mismatches with the XML size; swapped minimum and maximum values are corrected; boxes within a small configurable tolerance outside the image are clipped (and counted); larger overruns, non-numeric values, zero-area boxes and class codes outside the four supported codes are excluded and counted by reason; every exclusion appears in the conversion report. Unit tests check the formula against hand-computed boxes, and a visual check of converted labels on a sample is required.

```
import xml.etree.ElementTree as ET
from collections import Counter
import cv2

CODE_TO_ID = {"D00": 0, "D10": 1, "D20": 2, "D40": 3}
TOL_PX = 2.0                                  # configurable; report how many boxes get clipped

def convert_one(xml_path, img_path, rep: Counter):
    img = cv2.imread(str(img_path))
    if img is None:
        rep["unreadable_image"] += 1; return None
    H, W = img.shape[:2]
    root = ET.parse(xml_path).getroot()
    sz = root.find("size")
```

```
    if sz is not None:
        try:
            if (int(float(sz.findtext("width"))), int(float(sz.findtext("height")))) != (W, H):
                rep["xml_size_mismatch"] += 1
        except (TypeError, ValueError):
            rep["xml_size_missing"] += 1
    lines = []
    for obj in root.iter("object"):
        code = (obj.findtext("name") or "").strip()
        if code not in CODE_TO_ID:
            rep[f"excluded_code:{code}"] += 1; continue
        bb = obj.find("bndbox")
        try:
            x1, y1, x2, y2 = (float(bb.findtext(k)) for k in ("xmin", "ymin", "xmax", "ymax"))
        except (AttributeError, TypeError, ValueError):
            rep["bad_box_values"] += 1; continue
        x1, x2 = sorted((x1, x2)); y1, y2 = sorted((y1, y2))
        if x1 < -TOL_PX or y1 < -TOL_PX or x2 > W + TOL_PX or y2 > H + TOL_PX:
            rep["out_of_bounds"] += 1; continue
        cx1, cy1, cx2, cy2 = max(0.0, x1), max(0.0, y1), min(float(W), x2), min(float(H), y2)
        if (cx1, cy1, cx2, cy2) != (x1, y1, x2, y2):
            rep["clipped"] += 1
        if cx2 - cx1 < 1 or cy2 - cy1 < 1:
            rep["degenerate"] += 1; continue
        lines.append(f"{CODE_TO_ID[code]} {(cx1 + cx2) / 2 / W:.6f} {(cy1 + cy2) / 2 / H:.6f} "
                     f"{(cx2 - cx1) / W:.6f} {(cy2 - cy1) / H:.6f}")
    return lines
```

4.4 Dataset splitting and leakage prevention [REV]

- Only labelled images are split, at image level, into training, validation and testing sets; official unlabelled test images are kept separate and never used for quantitative evaluation.
- **Exact duplicates** are found by file hash. **Near-duplicates** are found by a perceptual difference hash; images within a small Hamming distance are merged into one group. The threshold is recorded, and the group-size distribution is inspected so unrelated road images are not over-merged.
- Splitting is done **by group**, stratified by the rarest class present in each image, with a recorded random seed (recommended ratio about 80 / 10 / 10; fixed after the inspection step shows real counts).
- The three lists are written to files and **frozen**; they are not regenerated after training starts.
- A script verifies that no group and no file hash appears in two splits; its result is stored in the PRD dataset record.
- Decide and record whether images without kept-class boxes are used as background examples. If used, they appear in all splits in natural proportion.
- Report actual image and object counts per split and per class.

The complete dataset is the acquisition target; the amount used for training is set by Section 4.6 and reported.

```
# training/split_dataset.py  (core functions)
import cv2, hashlib
from collections import defaultdict
from pathlib import Path
import numpy as np
from sklearn.model_selection import StratifiedGroupKFold

MAX_BUCKET = 2000      # skip pathological buckets (e.g. blank images) and log them

def sha1(p):
    return hashlib.sha1(Path(p).read_bytes()).hexdigest()

def dhash(p):
    g = cv2.imread(str(p), cv2.IMREAD_GRAYSCALE)
    if g is None:
        return None
    s = cv2.resize(g, (9, 8), interpolation=cv2.INTER_AREA)
    v = 0
    for bit in (s[:, 1:] > s[:, :-1]).flatten():
        v = (v << 1) | int(bit)
    return v

def group_near_duplicates(paths, max_hamming=3):
    parent = list(range(len(paths)))
    def find(i):
        while parent[i] != i:
            parent[i] = parent[parent[i]]; i = parent[i]
        return i
    hashes = [dhash(p) for p in paths]
    buckets = defaultdict(list)                      # pigeonhole: Hamming <= 3 over 64 bits
    for i, h in enumerate(hashes):                   # means one of four 16-bit chunks matches
        if h is not None:
            for k in range(4):
```

```
                buckets[(k, (h >> (16 * k)) & 0xFFFF)].append(i)
    for idxs in buckets.values():
        if len(idxs) > MAX_BUCKET:
            print("skipped oversized bucket:", len(idxs)); continue
        for a in range(len(idxs)):
            for b in range(a + 1, len(idxs)):
                i, j = idxs[a], idxs[b]
                if bin(hashes[i] ^ hashes[j]).count("1") <= max_hamming:
                    parent[find(i)] = find(j)
    return [find(i) for i in range(len(paths))]

def assign_splits(groups, rarest_class, seed=42):     # 10 folds -> 80 / 10 / 10
    sgkf = StratifiedGroupKFold(n_splits=10, shuffle=True, random_state=seed)
    folds = [set(te) for _, te in sgkf.split(np.zeros(len(groups)), rarest_class, groups)]
    return ["test" if i in folds[0] else "val" if i in folds[1] else "train" for i in range(len(groups))]

def assert_disjoint(groups, splits, hashes):
    seen_g, seen_h = {}, {}
    for g, s, h in zip(groups, splits, hashes):
        assert seen_g.setdefault(g, s) == s, "group spans two splits"
        assert seen_h.setdefault(h, s) == s, "duplicate file spans two splits"
```

4.5 YOLO dataset structure [REV]

```
dataset/
|-- images/{train,val,test}/
|-- labels/{train,val,test}/
`-- data.yaml

# dataset/data.yaml  (generated by split_dataset.py; absolute path written by the script)
path: C:/RoadVision/dataset
train: images/train
val: images/val
test: images/test
names:
  0: D00
  1: D10
  2: D20
  3: D40
```

data.yaml is generated by split_dataset.py with an **absolute** path (a relative path can be resolved against Ultralytics' own datasets directory rather than the working directory). *Optional:* write train.txt, val.txt and test.txt image lists instead of copying images into three folders; this saves disk and makes the frozen split auditable (verify on a ten-image test that label paths are derived correctly).

4.6 Training subset strategy [NEW]

Acquisition (the whole dataset) and training (the subset actually used) are reported separately.

- Fix and freeze the validation and test splits from the full labelled pool first; never subsample them for convenience.
- Subsample only the training split.
- Keep every training image that contains a rare class (read the real distribution from inspect_dataset.py) and subsample the dominant classes.
- Size the subset from measured time (formula in Section 5.2).
- Record the selection method, seed, per-class counts and the final subset size.

5. Model Training Pipeline [REV]

Step	Activity
1	Dataset acquisition and integrity check
2	Dataset inspection
3	Annotation conversion
4	Group-aware split, leakage check; freeze split lists
5	YOLO dataset configuration
6	GPU verification and staged smoke test (Section 5.2)
7	Pilot training of YOLO26n
8	Training of YOLO26m

Step	Activity
9	**Checkpoint and confidence-threshold selection on the validation set only**
10	Completion and selection for YOLO26n
11	**Single test-set evaluation of both final checkpoints** (the only use of the test set)
12	Inference integration and fallback tests

Figure 2: Training and integration pipeline (test evaluation follows validation-based selection)

5.1 Training considerations

- Use the NVIDIA GPU through the verified PyTorch CUDA build; the Section 14 gate must pass before training.
- Inspect GPU memory before choosing batch size.
- Use pretrained weights rather than training from scratch; save checkpoints every epoch and enable resume.
- Save training logs; record Ultralytics, PyTorch and driver versions.
- Never use the held-out test set for model, checkpoint or threshold selection.
- Run training with the application stopped; keep the laptop plugged in with its high-performance power mode; watch temperatures during long runs.
- On Windows, guard training scripts with if __name__ == '__main__', start with few dataloader workers, and do not cache the entire dataset in RAM (16 GB).

The design does not prescribe fixed epoch counts and does not guarantee training completion within seven days.

5.2 Staged smoke test and decision rules [NEW]

- **Stage 0 - GPU gate** (Section 14).
- **Stage 1 - data smoke test:** about 200 converted images, one epoch of YOLO26n; confirm in the training batch images that boxes and class order are correct.
- **Stage 2 - memory test for YOLO26m:** two epochs on the small set at image size 640 with batch 16, then 8, then 4; record peak GPU memory and wall time; choose the largest batch that finishes and leaves headroom. The values 640, 16, 8 and 4 are starting points of the ladder, not recommended settings.
- **Stage 3 - pilot:** about 10 percent of the intended training subset for a few epochs, to measure seconds per image per epoch and to see the metric trend.
- **Stage 4 - main run** with checkpoints every epoch and resume enabled.

Subset sizing: let t be the measured seconds per training image per epoch (including validation overhead), B the hours of GPU time the schedule allows for the model, and E the planned epochs. Maximum training images N = (B x 3600) / (t x E). Record t, B, E and N in the report.

YOLO26m go/no-go (Day 3): go if it fits in memory at batch 4 or more and N is large enough to include every rare-class image. Otherwise, in order: reduce image size, reduce batch, reduce the training subset. Only as a last resort is the primary model changed, and that must be recorded as a deviation from the PRD.

Setting	How it will be chosen
Image resolution	Smoke-test memory and speed; thin cracks can vanish at low resolution, so the trade-off is recorded
Batch size	Largest value that fits reliably in the Stage 2 test
Epochs	Validation behaviour and available time (formula above)
Initial weights	Official pretrained weights for the respective model
Training volume	Section 4.6 and the formula above; reported in the final report

Table 4: Training settings determined by measurement

```
# training/smoke_train.py   usage: python smoke_train.py yolo26m.pt 8 640
import sys, time
import torch
from ultralytics import YOLO

DATA_YAML_SMOKE = r"C:/RoadVision/dataset_smoke/data.yaml"

def main():
    weights, batch, imgsz = sys.argv[1], int(sys.argv[2]), int(sys.argv[3])
```

```
    torch.cuda.reset_peak_memory_stats()
    t0 = time.perf_counter()
    YOLO(weights).train(data=DATA_YAML_SMOKE, epochs=2, imgsz=imgsz, batch=batch, device=0,
                        workers=2, seed=0, project="runs_smoke", name=f"{weights}_{batch}_{imgsz}", exist_ok=True)
    dt = time.perf_counter() - t0
    print(f"wall {dt:.0f}s | peak allocated {torch.cuda.max_memory_allocated()/1e9:.2f} GB"
          f" | peak reserved {torch.cuda.max_memory_reserved()/1e9:.2f} GB")

if __name__ == "__main__":                    # required on Windows (dataloader worker processes)
    main()
```

6. Inference Engine [REV]

A shared inference service accepts an image or a frame and returns the structure below. The same interface serves image, video and webcam processing, so all three modes behave consistently. Class names are the codes; display labels come from classes.json.

```
{
  "model": "<active model name>",
  "total_detections": <integer>,                    // per-frame detections, not unique damage
  "class_counts": { "D00": <int>, "D10": <int>, "D20": <int>, "D40": <int> },
  "detections": [
    { "class_id": <int>, "class_name": "<code>", "confidence": <float>,
      "box": [x1, y1, x2, y2] }                     // original-image pixels
  ]
}
```

Listing: detection result structure (types only; implemented by to_result in Section 3.3)

The confidence threshold is configurable per request (PRD v1.1 FR-06; recommended default 0.25, range 0.10 to 0.90). The end-to-end or NMS mode (Section 3.4) is identical for both models. When YOLO26m and YOLO26n are compared, mode and thresholds are the same so differences reflect the models.

7. Image Processing Module [REV]

- Receive the upload as a stream to a temporary file; abort with 413 above the size limit.
- Verify with Pillow (format JPEG or PNG, pixel limit) and decode with OpenCV (Section 10.2).
- Create the inspection record, then pass the image through the inference lock to the detection engine.
- Generate the annotated image, calculate the class-wise summary, save the output under a generated name.
- Update the inspection record; delete the temporary upload; return the result with the output URL.

Invalid images and inference errors are handled safely: the request returns an understandable error with the shared error shape, an inspection record with status failed may be stored, and no partial output is presented as a result. If no damage is detected the result states that clearly.

8. Video Processing Module [REV]

- Receive the upload; validate size, duration, resolution and that it decodes with OpenCV (fps and frame count above zero).
- Create the inspection (queued), take the job slot (409 if busy), start a worker thread, return 202.
- Read frames sequentially; a frame cap is also enforced while reading because container metadata can be wrong.
- Run detection per frame through the model manager; update counts and status about once per second.
- Draw boxes and labels; pipe annotated frames to FFmpeg, which writes an **H.264 MP4 (yuv420p, faststart)** that plays in the browser; frames are padded to even dimensions.
- Honour cancel requests and the job time limit; on failure or cancel delete the partial output; always release the capture and encoder, delete the uploaded original, and free the slot.

Original frame rate is preserved where feasible (25 fps is assumed if the file reports none). The Day-1 gate confirms the bundled FFmpeg contains libx264; if it does not, another FFmpeg build is used, or output is download-only and the limitation is stated. Object tracking is optional; ByteTrack is not required, and counts are per-frame (PRD v1.1).

```
# backend/app/services/job_slot.py
import threading

class JobSlot:
```

```
    """One active video job or webcam session at a time."""
    def __init__(self):
        self._lock = threading.Lock()
        self.inspection_id = None
        self.kind = None
        self.cancel = threading.Event()

    def acquire(self, inspection_id, kind):
        with self._lock:
            if self.inspection_id is not None:
                return False
            self.inspection_id, self.kind = inspection_id, kind
            self.cancel.clear()
            return True

    def release(self):
        with self._lock:
            self.inspection_id = self.kind = None
            self.cancel.clear()

slot = JobSlot()

# backend/app/api/video.py  (skeleton)
import threading, time
from collections import Counter

# plain def, registered with @router.post("/detect/video", status_code=202)
def detect_video_endpoint(file, conf):
    path, display = save_and_validate_video(file)     # R-07: streamed save, limits, decodability
    iid = db_insert_inspection("video", display, conf, status="queued")
    if not slot.acquire(iid, "video"):
        db_mark(iid, "failed", "busy"); path.unlink(missing_ok=True)
        raise ApiError(409, "busy", "Another video job or camera session is running.")
    threading.Thread(target=run_video_job, args=(iid, path, conf), daemon=True).start()
    return {"inspection_id": iid, "status": "queued"}

def run_video_job(iid, src, conf):
    cap = proc = None; status, err = "failed", None
    out = OUTPUT_DIR / new_output_name("mp4"); counts = Counter(); done = hits = 0
    try:
        cap = cv2.VideoCapture(str(src))
        w, h = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
        fps = cap.get(cv2.CAP_PROP_FPS) or 25.0
        total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT)) or None
        proc = open_h264_writer(out, w, h, fps)       # R-05
        db_mark(iid, "processing", frames_total=total)
        deadline, last = time.monotonic() + JOB_TIMEOUT_S, 0.0
        while True:
            if slot.cancel.is_set():
                status = "cancelled"; break
            if time.monotonic() > deadline:
                raise TimeoutError("job time limit reached")
            ok, frame = cap.read()
            if not ok:
                status = "completed"; break
            res = models.predict(frame, conf)
            r = to_result(res, models.active)
            counts.update(r["class_counts"]); hits += 1 if r["total_detections"] else 0
            proc.stdin.write(res.plot().tobytes()); done += 1
            if time.monotonic() - last > 1.0:
                db_progress(iid, done, hits, counts); last = time.monotonic()
        proc.stdin.close()
        if proc.wait(timeout=120) != 0:
            raise RuntimeError("video encoder failed")
    except Exception as e:
        status, err = "failed", str(e)[:300]
    finally:
        if cap: cap.release()
        if proc and proc.poll() is None: proc.kill()
        if status != "completed": out.unlink(missing_ok=True)
        src.unlink(missing_ok=True)                    # originals not retained (PRD v1.1 FR-09)
        db_finish(iid, status, err, done, hits, counts, out.name if status == "completed" else None)
        slot.release()

import subprocess
import imageio_ffmpeg

def open_h264_writer(out_path, w, h, fps):
    cmd = [imageio_ffmpeg.get_ffmpeg_exe(), "-y", "-loglevel", "error",
           "-f", "rawvideo", "-pix_fmt", "bgr24", "-s", f"{w}x{h}", "-r", f"{fps:.3f}", "-i", "-",
           "-an", "-vf", "pad=ceil(iw/2)*2:ceil(ih/2)*2",
           "-c:v", "libx264", "-preset", "veryfast", "-crf", "23", "-pix_fmt", "yuv420p",
           "-movflags", "+faststart", str(out_path)]
    return subprocess.Popen(cmd, stdin=subprocess.PIPE, stderr=subprocess.DEVNULL)

# Day-1 check from a terminal (should list libx264):  <ffmpeg-exe> -hide_banner -encoders | findstr 264
```

9. Live Webcam Module [REV]

The module uses a locally connected webcam or USB camera; remote CCTV is not assumed. **Decision (PRD v1.1 FR-03): backend capture (Option B).** POST /api/camera/start opens the camera with OpenCV's DirectShow backend, takes the job slot and creates an inspection; a capture thread reads frames, runs the shared inference service and publishes the latest annotated JPEG; GET /api/camera/stream serves it as an MJPEG stream for an img element; POST /api/camera/stop ends the session.

Aspect	Design
Why Option B	No per-frame browser encoding or upload, fewer browser-permission variables, simpler cleanup; browser and backend run on the same laptop
Session end	Stop request; maximum duration (recommended 10 min); no stream consumer for the watchdog interval (recommended 10 s); camera stops returning frames; application shutdown
Cleanup	Capture thread releases the camera in a finally block, saves one aggregated inspection record, frees the job slot; shutdown hook calls stop()
Errors	No camera or camera in use: 503 camera_unavailable; disconnect mid-session: session ends, partial summary saved with a failure note; application keeps running
Live FPS and fallback	Rolling FPS shown in the interface; trigger F5 switches to YOLO26n with notice
Stored data	Aggregated per-frame counts only; frames are not stored

Table 5: Webcam design (the alternative browser-capture option is dropped to avoid duplicate camera routes)

```
# backend/app/services/camera.py  (skeleton)
import threading, time, cv2

class CameraService:
    def __init__(self, models, slot):
        self.models, self.slot = models, slot
        self._stop = threading.Event(); self._cond = threading.Condition()
        self._latest = None; self._last_client = 0.0; self._thread = None

    def start(self, index, conf):
        iid = db_insert_inspection("webcam", None, conf, status="processing")
        if not self.slot.acquire(iid, "webcam"):
            db_mark(iid, "failed", "busy"); raise ApiError(409, "busy", "A job is already running.")
        cap = cv2.VideoCapture(index, cv2.CAP_DSHOW)        # Windows DirectShow backend
        if not cap.isOpened():
            cap.release(); db_mark(iid, "failed", "camera unavailable"); self.slot.release()
            raise ApiError(503, "camera_unavailable", "Camera not found or in use.")
        self._stop.clear(); self._latest = None; self._last_client = time.monotonic()
        self._thread = threading.Thread(target=self._loop, args=(cap, iid, conf), daemon=True)
        self._thread.start()
        return iid

    def stop(self):
        self._stop.set()
        if self._thread:
            self._thread.join(timeout=5)

    def _loop(self, cap, iid, conf):
        status, err, started = "completed", None, time.monotonic()
        try:
            while not self._stop.is_set():
                if time.monotonic() - started > MAX_SESSION_S: break
                if time.monotonic() - self._last_client > WATCHDOG_S: err = "stream closed"; break
                ok, frame = cap.read()
                if not ok: status, err = "failed", "camera disconnected"; break
                res = self.models.predict(frame, conf)
                # update running counts, rolling FPS and trigger F5 here
                ok2, jpg = cv2.imencode(".jpg", res.plot(), [cv2.IMWRITE_JPEG_QUALITY, 80])
                with self._cond:
                    self._latest = jpg.tobytes(); self._cond.notify_all()
        except Exception as e:
            status, err = "failed", str(e)[:300]
        finally:
            cap.release()
            db_finish_webcam(iid, status, err)
            self.slot.release(); self._stop.set()

    def frames(self):                                   # generator for StreamingResponse
        last = None
        while not self._stop.is_set():
            with self._cond:
                self._cond.wait(timeout=1.0); jpg = self._latest
            self._last_client = time.monotonic()
            if jpg is not None and jpg is not last:
                last = jpg
```

```
            yield b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + jpg + b"\r\n"
```

The explicit stop route, the watchdog, the time limit and the shutdown hook are the guaranteed release paths; the design does not rely on the framework noticing a closed browser tab.

10. Backend API Design [REV]

FastAPI's interactive documentation documents the endpoints. Rules: every input type creates an inspection; one error shape for all failures; heavy handlers are plain def; detection routes take a conf parameter within the allowed range.

Method	Endpoint	Purpose	Success	Errors
GET	/api/health	Liveness	200	-
GET	/api/model/status	Active model, availability, fallback flag and reason, nms_mode	200	-
POST	/api/detect/image	Image detection (synchronous)	200	400, 413, 415, 422, 503, 500
POST	/api/detect/video	Start video job	202 + inspection_id	400, 409, 413, 415, 422, 503
POST	/api/history/{id}/cancel	Cancel a running job	200	404, 409
POST	/api/camera/start	Start webcam session	200 + inspection_id	409 busy, 503 camera
GET	/api/camera/stream	MJPEG stream of annotated frames	200 stream	409 no session
POST	/api/camera/stop	Stop session; returns summary	200	409 not running
GET	/api/history	List (limit, offset, range-checked)	200	422
GET	/api/history/{id}	One record; also used for job polling	200	404, 422
DELETE	/api/history/{id}	Delete record and output file	204	404, 409 running
GET	/api/dashboard	Totals, class totals, recent items, active model	200	-
GET	/api/results/{name}	Managed output file	200	400, 404

Table 6: Endpoints (replaces v1.0 Tables 6 and 7; the frame-detection alternative of v1.0 Section 10.2 is removed)

10.1 Concurrency, jobs, progress and cancellation [REV]

Video jobs and webcam sessions share one job slot: a second start receives 409 'busy'. Image requests are served one at a time through the inference lock (a request waits for at most one frame of a running job). A video upload returns 202 with an inspection id; the worker updates status, frames_processed and frames_total at most about once per second; the frontend polls GET /api/history/{id}. POST /api/history/{id}/cancel sets the slot's cancel event; the worker stops, deletes partial output and records 'cancelled'. At start-up any record left 'queued' or 'processing' is set to 'failed' with message 'interrupted by restart'. Jobs that fail leave a failed record and do not affect other sessions.

10.2 Upload handling, validation and managed files [NEW]

Uploads are streamed to a temporary file in chunks and abandoned with 413 once the limit is exceeded (a Content-Length header over the limit is rejected first; the framework may already have received the body, so this is not the only control). Client filenames are never used as paths; they are reduced to a display name. Files are stored under generated names. Images are verified with Pillow and decoded with OpenCV; videos must open with fps and frame count above zero and be within the limits below. /api/results/{name} accepts only names matching a fixed pattern, resolved inside the output directory. The database and API expose managed names and URLs, never absolute paths.

Limit (recommended; PRD v1.1 Section 4.11)	Value
Image	JPG, JPEG, PNG; up to 20 MB; longest side up to 8192 px
Video	MP4, AVI; up to 200 MB; up to 60 s; up to 1920 x 1080; job time limit 10 min

Limit (recommended; PRD v1.1 Section 4.11)	Value
Webcam session	Automatic stop after 10 min; minimum live FPS 5
Concurrency	One video job or webcam session at a time

```
# backend/app/utils/files.py
import re, uuid
from datetime import datetime
from pathlib import Path
from PIL import Image
import cv2

Image.MAX_IMAGE_PIXELS = MAX_IMAGE_SIDE * MAX_IMAGE_SIDE      # raises DecompressionBombError above this
SAFE_NAME = re.compile(r"^[A-Za-z0-9_-]{8,64}\.(jpg|png|mp4)$")

def display_name(raw):
    name = Path((raw or "upload").replace("\\", "/")).name
    return re.sub(r"[^\w.\- ]", "_", name)[:100] or "upload"

def new_output_name(ext):
    return f"{datetime.now():%Y%m%d_%H%M%S}_{uuid.uuid4().hex[:8]}.{ext}"

def save_upload(upload, dest, max_bytes):                 # sync: used from plain def handlers
    size = 0
    try:
        with dest.open("wb") as out:
            while chunk := upload.file.read(1024 * 1024):
                size += len(chunk)
                if size > max_bytes:
                    raise ApiError(413, "too_large", "File exceeds the size limit.")
                out.write(chunk)
    except Exception:
        dest.unlink(missing_ok=True); raise

def validate_image(path):
    try:
        with Image.open(path) as im:
            fmt, (w, h) = im.format, im.size
            im.verify()
    except Exception:
        raise ApiError(400, "undecodable_image", "The file is not a readable image.")
    if fmt not in ("JPEG", "PNG"):
        raise ApiError(415, "unsupported_type", "Only JPG, JPEG and PNG are supported.")
    if max(w, h) > MAX_IMAGE_SIDE:
        raise ApiError(422, "image_too_large", "Image dimensions exceed the limit.")
    img = cv2.imread(str(path))
    if img is None:
        raise ApiError(400, "undecodable_image", "The image could not be decoded.")
    return img

def validate_video(path):
    cap = cv2.VideoCapture(str(path))
    try:
        fps, n = cap.get(cv2.CAP_PROP_FPS), cap.get(cv2.CAP_PROP_FRAME_COUNT)
        w, h = cap.get(cv2.CAP_PROP_FRAME_WIDTH), cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
        ok, _ = cap.read()
    finally:
        cap.release()
    if not ok or fps <= 0 or n <= 0:
        raise ApiError(415, "unsupported_video", "The video could not be decoded.")
    if n / fps > MAX_VIDEO_SECONDS or max(w, h) > MAX_VIDEO_SIDE:
        raise ApiError(422, "video_over_limit", "Video duration or resolution exceeds the limit.")

# backend/app/api/results.py
def get_result(name):                                     # plain def; FileResponse from starlette
    if not SAFE_NAME.fullmatch(name):
        raise ApiError(400, "invalid_name", "Invalid file name.")
    path = (OUTPUT_DIR / name).resolve()
    if path.parent != OUTPUT_DIR.resolve() or not path.is_file():
        raise ApiError(404, "not_found", "Output no longer available.")
    return FileResponse(path)                             # verify in-browser seeking of mp4
```

10.3 Error shape and schemas [NEW]

All errors return {"error": {"code": "...", "message": "..."}}, including validation errors (422). Stack traces and filesystem paths are never returned.

```
# backend/app/schemas.py
from typing import Literal
from pydantic import BaseModel

Status = Literal["queued", "processing", "completed", "failed", "cancelled"]

class Detection(BaseModel):
    class_id: int
```

```
    class_name: str                     # code: D00, D10, D20, D40
    confidence: float
    box: list[float]                    # x1, y1, x2, y2 in original-image pixels

class InspectionOut(BaseModel):
    id: int
    created_at: str
    input_type: Literal["image", "video", "webcam"]
    filename: str | None
    models_used: str | None
    conf_threshold: float | None
    nms_mode: str | None
    total_detections: int
    class_summary: dict[str, int]
    frames_total: int | None
    frames_processed: int
    frames_with_detections: int
    processing_ms: int | None
    output_url: str | None              # "/api/results/<name>" or None
    status: Status
    error_message: str | None

class ImageResult(InspectionOut):
    detections: list[Detection]

# backend/app/errors.py
from fastapi import Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

class ApiError(Exception):
    def __init__(self, status, code, message):
        self.status, self.code, self.message = status, code, message

def install_handlers(app):
    @app.exception_handler(ApiError)
    async def _api(request: Request, exc: ApiError):
        return JSONResponse(status_code=exc.status, content={"error": {"code": exc.code, "message": exc.message}})
    @app.exception_handler(RequestValidationError)
    async def _val(request: Request, exc: RequestValidationError):
        return JSONResponse(status_code=422, content={"error": {"code": "invalid_parameters",
                                                                "message": "Invalid request parameters."}})
```

11. Database Design [REV]

SQLite is used for local storage in WAL mode. Image and video files stay in the filesystem; the database stores only managed file names, never absolute paths or large blobs. Every operation opens its own short-lived connection, because the video worker, the camera thread and request threads write and read concurrently. All SQL is parameterised; page sizes and IDs are validated before they reach the database. Timestamps are UTC ISO-8601 text. The dashboard aggregates completed records' class_summary JSON in Python (demonstration-scale data).

Field	Type	Description
id	INTEGER PK	Inspection ID
created_at, finished_at	TEXT	UTC timestamps
input_type	TEXT (CHECK)	image, video or webcam
filename	TEXT	Sanitised display name
models_used	TEXT	For example YOLO26m or YOLO26m,YOLO26n
conf_threshold, nms_mode, iou_threshold	REAL / TEXT	Settings used (iou only in NMS mode)
total_detections	INTEGER	Per-frame detections
class_summary	TEXT (JSON)	Counts per class code
frames_total, frames_processed, frames_with_detections	INTEGER	Video and webcam progress and summary
processing_ms	INTEGER	Processing duration
output_name	TEXT	Managed output file name
status	TEXT (CHECK)	queued, processing, completed, failed, cancelled
error_message	TEXT	Reason for failed or cancelled

Table 8: Inspections table (replaces v1.0; covers PRD v1.1 FR-08)

```
-- backend/app/database/schema.sql
PRAGMA journal_mode = WAL;
CREATE TABLE IF NOT EXISTS inspections (
  id                    INTEGER PRIMARY KEY AUTOINCREMENT,
  created_at            TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now')),
  finished_at           TEXT,
  input_type            TEXT NOT NULL CHECK (input_type IN ('image','video','webcam')),
  filename              TEXT,                           -- sanitised display name only
  models_used           TEXT,                           -- e.g. 'YOLO26m' or 'YOLO26m,YOLO26n'
  conf_threshold        REAL,
  nms_mode              TEXT CHECK (nms_mode IN ('end2end','nms')),
  iou_threshold         REAL,                           -- only when nms_mode = 'nms'
  total_detections      INTEGER NOT NULL DEFAULT 0,     -- per-frame detections
  class_summary         TEXT NOT NULL DEFAULT '{}',     -- JSON {"D00":0,"D10":0,"D20":0,"D40":0}
  frames_total          INTEGER,
  frames_processed      INTEGER NOT NULL DEFAULT 0,
  frames_with_detections INTEGER NOT NULL DEFAULT 0,
  processing_ms         INTEGER,
  output_name           TEXT,                           -- managed file name, never a path
  status                TEXT NOT NULL CHECK (status IN ('queued','processing','completed','failed','cancelled')),
  error_message         TEXT
);
CREATE INDEX IF NOT EXISTS idx_inspections_created ON inspections (created_at DESC);

# backend/app/database/db.py
import sqlite3
from contextlib import contextmanager

@contextmanager
def db():
    con = sqlite3.connect(DB_PATH, timeout=10)
    con.row_factory = sqlite3.Row
    try:
        yield con
        con.commit()
    except Exception:
        con.rollback(); raise
    finally:
        con.close()

def init_db():
    with db() as con:
        con.executescript(SCHEMA_SQL)
        con.execute("UPDATE inspections SET status='failed', error_message='interrupted by restart', "
                    "finished_at=strftime('%Y-%m-%dT%H:%M:%fZ','now') WHERE status IN ('queued','processing')")

def list_inspections(limit=20, offset=0):
    limit, offset = max(1, min(int(limit), 100)), max(0, int(offset))
    with db() as con:
        return [dict(r) for r in con.execute(
            "SELECT * FROM inspections ORDER BY id DESC LIMIT ? OFFSET ?", (limit, offset))]
```

12. Frontend Design [REV]

The frontend uses React, TypeScript, Vite and Tailwind CSS, with a clean, responsive interface suitable for a college hall demonstration.

Page	Content
Dashboard	Total inspections, total detections, class-wise counts, recent inspections, active model with fallback notice, links to detection pages; empty state when no records
Image Detection	Upload control, input preview, annotated result, confidence scores, class-wise summary, threshold control, download
Video Detection	Upload control, status and progress, cancel button, annotated preview (video element), class-wise summary, download
Live Camera	Start and stop controls, annotated live view, detected classes and confidence, live FPS, active model, camera-error message
Inspection History	List of past inspections; selecting one shows its summary and output or 'output no longer available'; delete

Table 9: Frontend pages

- Every detection screen shows loading, empty and error states; error messages come from the shared error shape.
- A shared API client and TypeScript types in src/services and src/types mirror the Pydantic schemas.

- Job progress uses a polling hook; the live view is an img element on the MJPEG stream; the stop sequence clears the img source first, then calls the stop endpoint.
- Confidence-threshold control uses the allowed range; the active threshold is shown with results.

```
// frontend/vite.config.ts  (follow the current Tailwind-for-Vite install guide for its plugin)
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
export default defineConfig({
  plugins: [react()],
  server: { proxy: { '/api': 'http://127.0.0.1:8000' } },   // no CORS needed in development
})

// frontend/src/services/useInspection.ts - poll a job until it reaches a final status
import { useEffect, useState } from 'react'
import type { Inspection } from '../types'
const FINAL = ['completed', 'failed', 'cancelled']
export function useInspection(id: number | null, intervalMs = 1000) {
  const [data, setData] = useState<Inspection | null>(null)
  useEffect(() => {
    if (id === null) return
    let stopped = false
    let timer = 0
    const tick = async () => {
      try {
        const r = await fetch(`/api/history/${id}`)
        if (r.ok) {
          const d: Inspection = await r.json()
          if (!stopped) setData(d)
          if (FINAL.includes(d.status)) return
        }
      } catch { /* backend busy or restarting: try again */ }
      if (!stopped) timer = window.setTimeout(tick, intervalMs)
    }
    tick()
    return () => { stopped = true; window.clearTimeout(timer) }
  }, [id, intervalMs])
  return data
}
// Live page: start -> POST /api/camera/start, then <img src="/api/camera/stream" />.
// Stop -> set the img src to '' first, then POST /api/camera/stop. Show errors from { error: { message } }.
// Demo build: npm run build, then serve frontend/dist from FastAPI StaticFiles (one port, one command).
```

13. Proposed Project Folder Structure [REV]

```
RoadVision/
|-- backend/
|   |-- app/
|   |   |-- main.py            # lifespan: init_db, models.startup, camera shutdown
|   |   |-- config.py          # limits, thresholds, paths, MODEL_PATHS, CLASS_CODES, NMS_ARGS
|   |   |-- classes.json
|   |   |-- schemas.py  errors.py
|   |   |-- api/  services/  database/  utils/
|   |-- tests/                 # pytest: converter, splitter, API (TestClient)
|   |-- requirements.txt       # pinned after the Day-1 gate
|-- frontend/                  # src/{components,pages,services,types}, package.json, .nvmrc
|-- training/                  # verify_gpu, inspect_dataset, convert_annotations, split_dataset,
|                              #   smoke_train, train, evaluate
|-- weights/{yolo26m,yolo26n}/ # checkpoints (git-ignored)
|-- data/{uploads_tmp,outputs,db}/   # all runtime files under one configurable DATA_DIR
|-- dataset/  docs/  README.md
```

This is a logical structure that may be adjusted. The model name is YOLO26 (official weight files yolo26m.pt and yolo26n.pt); v1.0's 'yolov26' folder names were corrected. Large datasets, generated videos, runtime data and model checkpoints are kept out of Git through .gitignore entries and are documented in the README.

14. Hardware and Software Requirements [REV]

Hardware (development)	Specification
Computer	ASUS ROG Zephyrus G14
Processor	AMD Ryzen 9
GPU	NVIDIA RTX 5060 Laptop GPU; VRAM and driver version to be recorded (nvidia-smi)
Memory	16 GB RAM
Storage	1 TB
Power	Plugged in, high-performance power mode during training

Table 10: Development hardware

Software	Requirement
Operating system	Windows 11
Python	3.11
Node.js	24.x (pinned in .nvmrc)
Deep learning	PyTorch CUDA build for CUDA 12.8 or newer (supports compute capability 12.0), installed from the official selector before Ultralytics; NVIDIA driver recent enough for that build
Libraries	Ultralytics, OpenCV, imageio-ffmpeg, FastAPI, Uvicorn, Pillow, scikit-learn, pytest; React, TypeScript, Vite, Tailwind CSS; SQLite

Table 11: Software

Installation order and gate [NEW]: install the CUDA build of PyTorch first, then Ultralytics, then confirm that torch was not replaced by a CPU-only build. Run training/verify_gpu.py on Day 1; training and development do not proceed until it prints GATE PASSED. A plain pip install of torch on Windows can give a CPU-only build, and builds for older CUDA versions contain no kernels for this GPU. Unofficial PyTorch builds are not used. Pin all versions (pip freeze, .nvmrc, package-lock) after the gate and record them in the report. A separate CUDA toolkit is installed only if the chosen setup requires it.

```
# training/verify_gpu.py  - Day-1 gate. Syntax-checked only; run it on the project laptop.
import sys
import numpy as np
import torch

print("python", sys.version.split()[0], "| torch", torch.__version__, "| built for CUDA", torch.version.cuda)
assert torch.cuda.is_available(), "CUDA unavailable: CPU-only torch build or driver problem"
cc = torch.cuda.get_device_capability(0)
arch = torch.cuda.get_arch_list()
need_sm, need_ptx = f"sm_{cc[0]}{cc[1]}", f"compute_{cc[0]}{cc[1]}"
print(torch.cuda.get_device_name(0), "capability", cc, "arch list", arch)
assert need_sm in arch or need_ptx in arch, f"This torch build has no kernels for {need_sm}"
x = torch.randn(2048, 2048, device="cuda")
y = x @ x
torch.cuda.synchronize()
print("matmul ok, finite:", bool(torch.isfinite(y).all()))
print("VRAM total (GB):", round(torch.cuda.get_device_properties(0).total_memory / 1e9, 1))

from ultralytics import YOLO          # imported after torch on purpose
for name in ("yolo26n.pt", "yolo26m.pt"):
    m = YOLO(name)                    # downloads official weights on first use
    r = m.predict(np.zeros((640, 640, 3), np.uint8), device=0, imgsz=640, verbose=False)
    print(name, "forward pass ok; classes in COCO head:", len(m.names))
import cv2
print("opencv", cv2.__version__)
import imageio_ffmpeg
print("ffmpeg exe:", imageio_ffmpeg.get_ffmpeg_exe())   # then confirm libx264 (see R-05)
print("GATE PASSED")
```

15. Error Handling and Security [REV]

Area	Design measure
Uploads	Streamed size cap (413); content-based type check (415); image pixel limit; video duration, resolution and decodability checks; client filename never used as a path; generated storage names
Models	Load, class and warm-up checks; fallback triggers F1 to F6; clear 503 when no model is available; low confidence never treated as failure
Video jobs	Background thread; progress; cancel; time limit; restart once on fallback; partial output deleted on failure or cancel; uploaded original deleted after processing
Camera	DirectShow capture; clear errors for missing, busy or disconnected camera; watchdog, time limit and guaranteed release
Database	Parameterised SQL; CHECK constraints; WAL; per-operation connections; restart recovery of unfinished jobs
Resources	Cleanup of capture and encoder handles, camera handles and temporary files at job end and at start-up
Network	Backend bound to 127.0.0.1; no external calls during inference; CORS avoided by proxy or same-origin serving; no authentication (single-user local prototype)
API responses	No filesystem paths or stack traces; results addressed by validated managed names only; one error shape

Area	Design measure
Logging	Rotating local log file with timestamps: model loading, fallback activation, job failures

Table 12: Error handling and security measures

16. Performance, Evaluation and Testing [REV]

Evaluation uses held-out labelled test data, evaluated once per final checkpoint, after validation-based selection. YOLO26m and YOLO26n are compared on the same test set with identical settings (including end-to-end or NMS mode). All values are to be measured; none are stated in this document.

Measure	Basis and definition	Value
Precision, recall	Test set, overall and per class. Ultralytics' summary values correspond to its own best-F1 operating point (confirm for the installed version); also report values at the documented operating threshold chosen on the validation set	To be measured
mAP@0.5, mAP@0.5:0.95	Test set, overall and per class (conf=0.001 for mAP computation)	To be measured
Image inference time	Median and mean over a stated number of runs after warm-up; model time, and end-to-end separately	To be measured
Video processing FPS	Frames processed / wall-clock time for a stated clip	To be measured
Live-camera performance	Average displayed FPS and approximate latency over a stated session, per model	To be measured
GPU memory usage	Peak allocated and reserved memory during training smoke test and inference	To be measured

Table 13: Evaluation measures (values to be measured)

```
# training/evaluate.py   usage: python evaluate.py <best.pt> <data.yaml> <name>
import json, sys
from ultralytics import YOLO

def main():
    weights, data, name = sys.argv[1:4]
    m = YOLO(weights)
    r = m.val(data=data, split="test", imgsz=IMGSZ, conf=0.001, device=0, plots=True,
              project="eval", name=name, exist_ok=False, **NMS_ARGS)
    out = {"weights": weights, "map50": float(r.box.map50), "map50_95": float(r.box.map),
           "precision_summary": float(r.box.mp), "recall_summary": float(r.box.mr),
           "per_class_map50_95": {m.names[int(i)]: float(r.box.maps[int(i)]) for i in r.box.ap_class_index},
           "speed_ms": r.speed}
    json.dump(out, open(f"eval/{name}.json", "w"), indent=2)
    print(json.dumps(out, indent=2))

if __name__ == "__main__":
    main()
```

16.1 Testing strategy [NEW]

- **Unit tests (pytest):** coordinate conversion against hand-computed boxes; class-code mapping; splitter disjointness; filename sanitiser; results-route name validation.
- **API tests (FastAPI TestClient):** invalid type, oversize, corrupt image; busy (409); cancel; error shape; history paging limits.
- **Gate script:** verify_gpu.py on Day 1 and before the demonstration.
- **Failure injection:** rename the YOLO26m checkpoint; rename both; unplug the camera; supply an unsupported video.
- **Acceptance:** manual checklist mapped to PRD v1.1 AC-01 to AC-19, with results recorded in the report.

17. Seven-Day Technical Milestones [REV]

Day	ML track (suggested: Ram)	App track (suggested: Shakib)	Gate
1	Environment; verify_gpu.py; start RDD2022 download and integrity check; inspect_dataset.py	Repo, FastAPI skeleton, config, schema, model manager with stock weights, Vite proxy	Day-1 gate: GPU script passes; both weights run; libx264 present

Day	ML track (suggested: Ram)	App track (suggested: Shakib)	Gate
2	Conversion, grouping, frozen split, data.yaml; smoke tests; start YOLO26n pilot training	Image endpoint end to end: uploads, validation, history, results route	Split disjoint check passes
3	Smoke-test YOLO26m settings; start YOLO26m training (long run, app stopped)	Video job, progress, cancel, FFmpeg writer; image and video pages (pilot weights)	YOLO26m go/no-go
4	Complete YOLO26n; validation-based selection for n	Camera service, live page, dashboard, history, delete	Webcam start/stop/cleanup works
5	YOLO26m finishes; selection for m; single test evaluation of both; benchmarks	Fallback tests, error states, negative tests	Metrics recorded
6	Evaluation write-up; per-class analysis	Acceptance testing AC-01 to AC-19; fixes; README	Checklist complete
7	Report and figures	Demo rehearsal; label incomplete features as incomplete	Demonstration

Table 14: Technical milestones

The complete dataset download and processing overlaps with development; dataset preparation must not block application work, which proceeds against a small sample set and then the YOLO26n pilot weights while the full data is acquired and the long training run is in progress.

18. Licensing and References [REV]

[1] RDD2022 dataset: https://doi.org/10.6084/m9.figshare.21431547.v1

[2] RDD2022 research paper: https://doi.org/10.1002/gdj3.260

[3] YOLO26 documentation: https://docs.ultralytics.com/models/yolo26

[4] PyTorch: https://pytorch.org/

[5] FastAPI: https://fastapi.tiangolo.com/

[6] OpenCV: https://opencv.org/

[7] React: https://react.dev/

[8] SQLite: https://sqlite.org/

[9] Ultralytics licensing: https://www.ultralytics.com/license

[10] Vite: https://vite.dev/

[11] Tailwind CSS: https://tailwindcss.com/

The RDD2022 dataset will be attributed in the final report and README. Ultralytics software and models are offered under AGPL-3.0 and an Enterprise licence; the project's own licence choice is stated in the README and the terms of the dataset (check its figshare page), the YOLO26 models, PyTorch and all other dependencies are reviewed and complied with before anything is distributed. Dataset files and derived weights are not redistributed unless their licences allow it.

Appendix A: Change Log (v1.0 to v1.1) [NEW]

Section	Change and reason
2, 14	Tailwind added; one worker, 127.0.0.1; FFmpeg component; serving model (Vite proxy, static serving); PyTorch CUDA build requirement for the RTX 5060, install order, Day-1 gate script; VRAM, driver, power-mode rows.
3.3, 3.4	Fallback triggers F1 to F6 aligned with PRD v1.1; class check and warm-up; model manager with retry and runtime switch; YOLO26 end-to-end (NMS-free) mode decision.
4.1 to 4.6	Integrity checks; conversion rules with tolerance and report; class codes as model names; group-aware split with near-duplicate detection and frozen lists; absolute yaml path; training-subset strategy.
5	Pipeline reordered: validation-based selection before the single test evaluation; YOLO26n pilot first; staged smoke test, decision rules, subset formula; Windows precautions.
6	Result structure uses class codes; per-frame meaning of counts; threshold range and mode.
7, 8	Streamed validation; H.264 via FFmpeg; job slot, cancel, timeout, cleanup.
9	Webcam Option B decided; watchdog, time limit, guaranteed release; Option A removed.
10	Uniform API (cancel, delete, dashboard, stream); concurrency design; upload handling; error shape and schemas; limits table; v1.0 Section 10.2 removed.
11	Schema replaced to meet PRD v1.1 FR-08; WAL, per-operation connections, restart recovery.
12, 13	Frontend integration (types, polling, MJPEG, proxy); folder structure corrected (yolo26 naming, config, tests, data directory).
15, 16, 16.1	Expanded security and error table; evaluation definitions and script; testing strategy.
17	Milestones re-sequenced around training; two-track plan with gates.
18	Licensing detail (AGPL-3.0 and Enterprise).

Appendix B: Decisions for the Team to Approve [NEW]

- Operating limits and thresholds in Section 10.2 (recommended values, to be re-checked after the first throughput measurement).
- End-to-end versus NMS mode (recommended: end-to-end default).
- Whether annotation-free images are used as background examples.
- Perceptual-hash distance and split ratio (recommended about 80 / 10 / 10).
- Minimum live FPS for trigger F5 (recommended 5) and the watchdog interval.
- Optional: image-list files instead of copied folders; manual model selector; FP16 inference; ONNX Runtime and ByteTrack remain optional and out of the core scope.

Appendix C: Items Not Verified [NEW]

- RTX 5060 VRAM, training time, accuracy on RDD2022, RDD2022 size and class distribution.
- libx264 availability in the bundled FFmpeg, browser seeking of served MP4 files, Ultralytics handling of txt image lists.
- Exact PyTorch, Ultralytics, Vite and Tailwind version compatibility with Python 3.11 and Node 24; verify at install time and pin.
- Consistency with the Project Proposal (not provided).